Automated SOAR Playbook

```
#!/usr/bin/env python3
"""
Automated SOAR Playbook — safe Shuffle webhook integration template.

Receives a normalized security event (stdin or JSON file), evaluates a
conservative set of indicators, and can optionally request a Shuffle workflow
webhook. It does NOT directly isolate endpoints. Endpoint isolation must be
implemented as a gated action inside a reviewed Shuffle workflow, with an
allowlist, approval/verification steps, and a rollback path.

Use only in systems and environments you are authorized to administer.
"""
from __future__ import annotations
import argparse
import json
import os
import sys
from datetime import datetime, timezone
from pathlib import Path
from urllib.error import HTTPError, URLError
from urllib.parse import urlparse
from urllib.request import Request, urlopen

def load_event(path: str | None) -> dict:
    try:
        raw = Path(path).read_text(encoding="utf-8") if path else sys.stdin.read()
    except OSError as exc:
        raise SystemExit(f"Unable to read event input: {exc}")
    try:
        event = json.loads(raw)
    except json.JSONDecodeError as exc:
        raise SystemExit(f"Invalid event JSON: {exc}")
    if not isinstance(event, dict):
        raise SystemExit("Event must be a JSON object.")
    return event

def evaluate_event(event: dict, threshold: int = 8) -> dict:
    """Create a triage decision; this is a demonstration rule, not a verdict."""
    try:
        severity = int(event.get("severity", 0))
    except (TypeError, ValueError):
        severity = 0
    event_type = str(event.get("event_type", "unknown")).lower()
    known_bad = bool(event.get("known_malicious", False))
    # Conservative gating: require a strong explicit indicator or high severity
    # plus an event type on a small demonstration allowlist.
    eligible_types = {"malware_detected", "edr_detection", "credential_theft_indicator"}
    candidate = known_bad or (severity >= threshold and event_type in eligible_types)
    endpoint = str(event.get("endpoint_id", "")).strip()
    decision = "REVIEW_REQUIRED" if candidate else "NO_AUTOMATION"
    if candidate and not endpoint:
        decision = "REVIEW_REQUIRED_MISSING_ENDPOINT"
    return {
        "event_id": str(event.get("event_id", "unknown")),
        "received_at_utc": datetime.now(timezone.utc).isoformat(),
        "severity": severity,
        "event_type": event_type,
        "endpoint_id": endpoint or None,
        "automation_candidate": candidate,
        "decision": decision,
        "message": (
            "Candidate event requires Shuffle workflow policy checks and approval before any containment
action."
            if candidate else
            "No automated containment requested by the demonstration rule."
        ),
        "safety_note": "This script never isolates an endpoint. Implement containment only in a reviewed,
 authenticated, allowlisted Shuffle workflow."
    }

def notify_shuffle(webhook_url: str, decision: dict, timeout: float = 10.0) -> dict:
    parsed = urlparse(webhook_url)
    if parsed.scheme != "https" or not parsed.netloc:
        raise ValueError("Shuffle webhook URL must use HTTPS.")
    payload = json.dumps(decision).encode("utf-8")
    req = Request(webhook_url, data=payload, headers={
        "Content-Type": "application/json",
```

```
        "User-Agent": "AutomatedSOARPlaybook/1.0"
    }, method="POST")
    try:
        with urlopen(req, timeout=timeout) as response:
            return {"sent": True, "http_status": response.status}
    except HTTPError as exc:
        return {"sent": False, "http_status": exc.code, "error": f"HTTP {exc.code}"}
    except (URLError, TimeoutError, OSError) as exc:
        return {"sent": False, "error": str(exc)}

def main() -> int:
    parser = argparse.ArgumentParser(description="Evaluate a security event and optionally notify a
Shuffle HTTPS webhook.")
    parser.add_argument("--input", help="JSON event file; if omitted, read JSON from stdin.")
    parser.add_argument("--threshold", type=int, default=8, help="Severity threshold for allowlisted
event types.")
    parser.add_argument("--webhook-env", default="SHUFFLE_WEBHOOK_URL", help="Environment variable
containing the HTTPS webhook URL.")
    parser.add_argument("--notify", action="store_true", help="Send the decision to the configured
Shuffle webhook.")
    parser.add_argument("--output", default="soar_decision.json", help="Output decision JSON file.")
    args = parser.parse_args()
    if args.threshold < 0:
        parser.error("--threshold must be non-negative")
    event = load_event(args.input)
    decision = evaluate_event(event, args.threshold)
    if args.notify:
        webhook = os.environ.get(args.webhook_env, "")
        if not webhook:
            parser.error(f"Set {args.webhook_env} to your authenticated HTTPS Shuffle webhook URL.")
        try:
            decision["shuffle_notification"] = notify_shuffle(webhook, decision)
        except ValueError as exc:
            parser.error(str(exc))
    Path(args.output).write_text(json.dumps(decision, indent=2), encoding="utf-8")
    print(json.dumps(decision, indent=2))
    print(f"Decision written to {Path(args.output).resolve()}", file=sys.stderr)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```